

HANDS ON 1

Spring Data JPA Quick Example

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<parent>
```

```
  <groupId>org.springframework.boot</groupId>
```

```
  <artifactId>spring-boot-starter-parent</artifactId>
```

```
  <version>2.7.18</version>
```

```
</parent>
```

```
<groupId>com.cognizant</groupId>
```

```
<artifactId>orm-learn</artifactId>
```

```
<version>0.0.1-SNAPSHOT</version>
```

```
<properties>
```

```
  <java.version>1.8</java.version>
```

```
</properties>
```

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>org.springframework.boot</groupId>
```

```
    <artifactId>spring-boot-starter</artifactId>
```

```
  </dependency>
```

```
  <dependency>
```

```
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-data-jpa</artifactId>  
</dependency>
```

```
<dependency>  
  <groupId>mysql</groupId>  
  <artifactId>mysql-connector-java</artifactId>  
  <version>8.0.33</version>  
</dependency>
```

```
</dependencies>  
</project>
```

application.properties

```
logging.level.org.springframework=info  
logging.level.com.cognizant=debug
```

```
logging.level.org.hibernate.SQL=trace  
logging.level.org.hibernate.type.descriptor.sql=trace
```

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn  
spring.datasource.username=root  
spring.datasource.password=root
```

```
spring.jpa.hibernate.ddl-auto=validate  
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL5Dialect  
spring.jpa.show-sql=true
```

Create Database

```
create database ormlearn;
```

```
use ormlearn;
```

```
create table country(  
    co_code varchar(2) primary key,  
    co_name varchar(50)  
);
```

```
insert into country values('IN','India');  
insert into country values('US','United States');
```

Country Entity

```
package com.cognizant.ormlearn.model;
```

```
import javax.persistence.*;
```

```
@Entity
```

```
@Table(name="country")
```

```
public class Country {
```

```
    @Id
```

```
    @Column(name="co_code")
```

```
    private String code;
```

```
    @Column(name="co_name")
```

```
    private String name;
```

```
    public Country() {}
```

```
    public Country(String code,String name){
```

```
        this.code=code;
```

```
        this.name=name;
```

```
    }
```

```
    public String getCode(){
```

```
        return code;
```

```
    }
```

```
    public void setCode(String code){
```

```
        this.code=code;
```

```
    }
```

```
    public String getName(){
```

```
        return name;
```

```

    }

    public void setName(String name){
        this.name=name;
    }

    @Override
    public String toString(){
        return "Country [code="+code+
            ", name="+name+"]";
    }
}

```

Repository

```

package com.cognizant.ormlearn.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.cognizant.ormlearn.model.Country;

@Repository
public interface CountryRepository
    extends JpaRepository<Country,String>{

    List<Country> findByNameContaining(String name);
}

```

Exception

```

package com.cognizant.ormlearn.exception;

public class CountryNotFoundException
    extends Exception{

    public CountryNotFoundException(
        String msg){
        super(msg);
    }
}

```

Service Class

```
package com.cognizant.ormlearn.service;

import java.util.List;
import java.util.Optional;

import javax.transaction.Transactional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.cognizant.ormlearn.exception.CountryNotFoundException;
import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;

@Service
public class CountryService {

    @Autowired
    private CountryRepository countryRepository;

    @Transactional
    public List<Country> getAllCountries(){
        return countryRepository.findAll();
    }

    @Transactional
    public Country findCountryByCode(
        String code)
        throws CountryNotFoundException{

        Optional<Country> result=
            countryRepository.findById(code);

        if(!result.isPresent())
            throw new CountryNotFoundException(
                "Country not found");

        return result.get();
    }
}
```

```
@Transactional
public void addCountry(
    Country country){

    countryRepository.save(country);
}
```

```
@Transactional
public void updateCountry(
    String code,
    String name)
    throws CountryNotFoundException{

    Country country=
        findCountryByCode(code);

    country.setName(name);

    countryRepository.save(country);
}
```

```
@Transactional
public void deleteCountry(
    String code){

    countryRepository.deleteById(code);
}
```

```
@Transactional
public List<Country>
    searchCountry(
        String name){

    return countryRepository
        .findByNameContaining(name);
}
}
```

Main Class (Hands-on 1,5,6,7,8,9)

```
package com.cognizant.ormlearn;

import java.util.List;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;

@SpringBootApplication
public class OrmLearnApplication {

    private static CountryService service;

    public static void main(String[] args)
        throws Exception {

        ApplicationContext context=
            SpringApplication.run(
                OrmLearnApplication.class,
                args);

        service=
            context.getBean(
                CountryService.class);

        testGetAllCountries();

        testFindCountry();

        testAddCountry();

        testUpdateCountry();

        testDeleteCountry();

        testSearchCountry();
    }
}
```

```
private static void testGetAllCountries(){
```

```
    System.out.println(  
        "GET ALL");
```

```
    List<Country> countries=  
        service.getAllCountries();
```

```
    countries.forEach(  
        System.out::println;  
    )
```

```
private static void testFindCountry()  
    throws Exception{
```

```
    System.out.println(  
        service.findCountryByCode(  
            "IN"));
```

```
private static void testAddCountry(){
```

```
    service.addCountry(  
        new Country(  
            "SA",  
            "Saudi Arabia"));
```

```
    System.out.println(  
        "Country Added");
```

```
private static void testUpdateCountry()  
    throws Exception{
```

```
    service.updateCountry(  
        "SA",  
        "Saudi");
```

```
    System.out.println(  
        "Country Updated");
```



```

private static void testDeleteCountry(){

    service.deleteCountry(
        "SA");

    System.out.println(
        "Country Deleted");
}

private static void testSearchCountry(){

    List<Country> countries=
        service.searchCountry(
            "Uni");

    countries.forEach(
        System.out::println);
}
}

```

HANDS ON 2

Hibernate XML Configuration Example

hibernate.cfg.xml

```

<hibernate-configuration>
<session-factory>

<property name="hibernate.connection.driver_class">
com.mysql.cj.jdbc.Driver
</property>

<property name="hibernate.connection.url">
jdbc:mysql://localhost:3306/ormlearn
</property>

<property name="hibernate.connection.username">
root
</property>

<property name="hibernate.connection.password">

```

```
root
</property>

<mapping resource="Country.hbm.xml"/>

</session-factory>
</hibernate-configuration>
```

Country.hbm.xml

```
<hibernate-mapping>

<class name="Country"
      table="country">

  <id name="code"
      column="co_code"/>

  <property name="name"
      column="co_name"/>

</class>

</hibernate-mapping>
```

Hibernate Main

```
SessionFactory factory=
    new Configuration()
      .configure()
      .buildSessionFactory();

Session session=
    factory.openSession();

Transaction tx=
    session.beginTransaction();

session.save(
    new Country(
        "FR",
        "France"));
```

```
tx.commit();
```

```
session.close();
```

HANDS ON 3

Hibernate Annotation Configuration

```
@Entity
```

```
@Table(name="country")
```

```
public class Country {
```

```
    @Id
```

```
    @Column(name="co_code")
```

```
    private String code;
```

```
    @Column(name="co_name")
```

```
    private String name;
```

```
}
```

HANDS ON 5

Manage Country Services

Implemented:

```
getAllCountries()
```

```
indCountryByCode()
```

```
addCountry()
```

```
updateCountry()
```

```
deleteCountry()
```

```
searchCountry()
```

HANDS ON 6

Find Country By Code

```
Country country=
```

```
    service.findCountryByCode(  
        "IN");
```

```
System.out.println(country);
```

HANDS ON 7

Add Country

```
Country country=
    new Country(
        "SA",
        "Saudi Arabia");

service.addCountry(country);
```

HANDS ON 8

Update Country

```
service.updateCountry(
    "SA",
    "Saudi");
```

HANDS ON 9

Delete Country

```
service.deleteCountry(
    "SA");
```









